Nottingham Trent University
Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 3: Evolutionary Computation and Genetic Algorithms

Dr Pedro Machado pedro.machado@ntu.ac.uk

Nottingham Trent University

Department of Computer Science

Lecture Outline

In this lecture will cover:

- Computational Intelligence and Evolutionary Computation
- Motivation: Black-Box, Non-Convex, and Noisy Optimisation
- Evolutionary Computation: Core Principles and Taxonomy
- Genetic Algorithms
  - Representation, Selection, Crossover, Mutation
  - Advanced and Multi-Objective Variants
- Evolution Strategies
  - Self-Adaptation and Continuous Optimisation
  - CMA-ES
- Related Population-Based Methods
  - Differential Evolution
  - Genetic Programming
- Comparative Analysis and Complementarity
  - GA vs ES vs Swarm Methods
  - Hybrid Approaches
- Modern Applications and Research Directions

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

Evolutionary Computation and Genetic Algorithms

Evolutionary Computation (EC)

- Family of population-based stochastic optimisation methods
- Inspired by biological evolution and natural adaptation
- Operates on a set of candidate solutions evaluated by a fitness function
- Suitable for black-box, non-linear, and high-dimensional problems

Genetic Algorithms (GA)

- A core subclass of Evolutionary Computation
- Solutions encoded as chromosomes composed of genes
- Search driven by selection, crossover, and mutation
- Particularly effective for discrete and combinatorial optimisation

NTU

Why Evolutionary Computation

Black-Box Optimisation

- Optimise an unknown objective function:
  minimise $f(x)$, $x \in X$
- No analytical form or gradient available:
  gradient $f(x)$ unknown or unreliable
- Function evaluated only by sampling or simulation:
  $y = f(x) + \varepsilon$

Non-Convex and Noisy Problems

- Multiple local optima exist in the search space
- Objective function is not convex and may be discontinuous
- Noise affects evaluations:
  $f_{obs}(x) = f(x) + \varepsilon$, $\varepsilon \sim N(0, \sigma^2)$
- Optimisation targets robustness rather than exact optimality:
  minimise expected value $E[f(x)]$

NTU

Evolutionary Computation

Population-Based Optimisation

- Maintain a set of candidate solutions rather than a single point
- Population at iteration t: $P(t) = \{ x_1(t), x_2(t), \ldots, x_n(t) \}$
- Enables parallel exploration of multiple regions of the search space
- Reduces sensitivity to local optima and noisy evaluations

Fitness-Driven Search

- Each candidate solution evaluated using a fitness function f(x)
- Selection pressure biases search toward higher-quality solutions
- Probability of reproduction or survival increases with fitness:
  higher f(x) ⇒ higher selection likelihood
- Iterative improvement emerges from variation and selection

NTU

EC Taxonomy

Paradigm	Representation	Information Sharing	Main Operators	Strengths	Limitations	Typical Use Cases
Genetic Algorithms (GA)	Discrete / mixed encodings (binary, permutation, real)	Genetic recombination across population	Selection, crossover, mutation	Strong diversity preservation, flexible encoding, multi-objective optimisation	Slower convergence, parameter sensitivity	Feature selection, scheduling, combinatorial optimisation
Evolution Strategies (ES)	Real-valued vectors	Statistical adaptation via population ranking	Gaussian mutation, self-adaptation	Robust continuous optimisation, noise tolerance, efficient refinement	Computationally expensive in high dimensions	Engineering design, control, hyperparameter tuning
Swarm Intelligence	Real-valued or discrete (problem-dependent)	Social learning (global/local best, pheromones)	Velocity updates, probabilistic construction	Fast convergence, simple implementation, dynamic adaptation	Risk of stagnation, weaker diversity control	Routing, dynamic optimisation, real-time systems
Differential Evolution (DE)	Real-valued vectors	Vector differences between individuals	Difference-based mutation	Fast convergence, low algorithmic complexity	Sensitive to scaling parameters	Continuous optimisation
Genetic Programming (GP)	Program trees or graphs	Structural recombination	Tree crossover and mutation	Automated model discovery	High computational cost	Symbolic regression, rule discovery
Hybrid EC Systems	Mixed	Combined evolutionary and swarm mechanisms	Adaptive hybrids	Balanced speed, precision, and robustness	Design complexity	Large-scale, dynamic, real-world problems

NTU

Common Components of Evolutionary Computation

Set of candidate solutions evaluated in parallel

- Population at iteration t: $P(t) = \{ x_1(t), x_2(t), \ldots, x_n(t) \}$
- Enables exploration of multiple regions of the search space
- Improves robustness to noise and local optima

Representation

- Encoding of candidate solutions in a search space
- Can be binary, integer, permutation-based, real-valued, or structural
- Determines admissible variation operators and search bias

Fitness Function

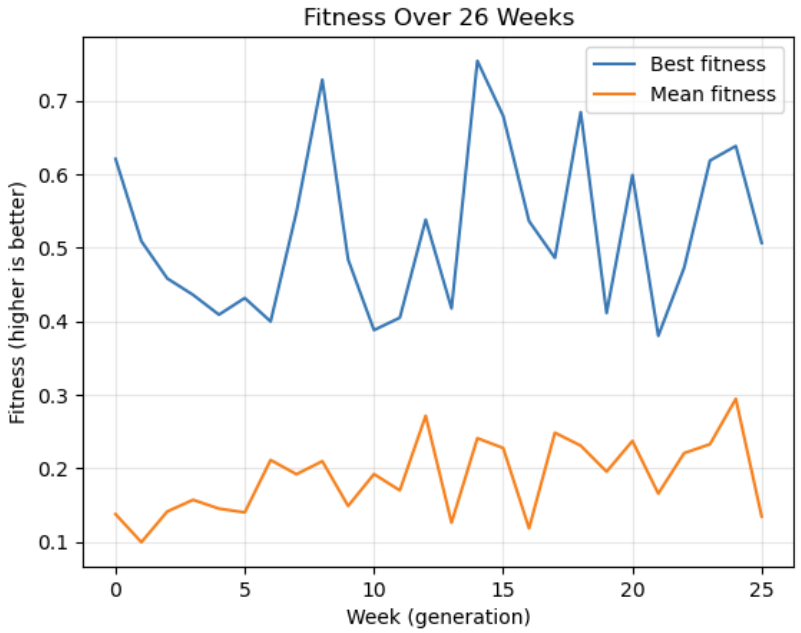
- Quantitative measure of solution quality
- Maps solutions to scalar values: $f : X \rightarrow \mathbb{R}$
- May include penalties for constraints or multiple objectives

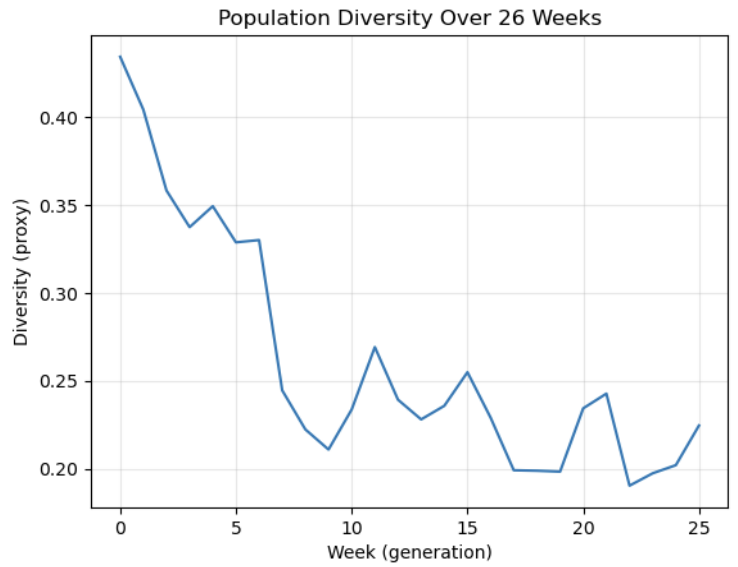
Variation Operators

- Generate new candidate solutions from existing ones
- Mutation introduces stochastic perturbations
- Crossover or recombination combines information from parents

Selection and Replacement

- Selection biases reproduction toward fitter individuals
- Replacement defines how offspring form the next population
- Controls selection pressure, diversity, and convergence rate





Genetic Algorithms: Overview

Origins and Motivation

- Inspired by Darwinian evolution and natural selection
- Introduced to solve optimisation problems where exhaustive or gradient-based methods are impractical
- Designed to explore large, complex search spaces with minimal problem assumptions

Discrete Optimisation Focus

- Naturally suited to discrete, combinatorial, and mixed-variable problems
- Common applications include feature selection, scheduling, routing, and subset optimisation
- Encoding-based search allows constraints and structure to be embedded in chromosomes

Search via Recombination

- Evolution driven by selection and genetic recombination
- Crossover exchanges building blocks between solutions
- Mutation maintains diversity and prevents premature convergence
- Global search emerges from population-level recombination rather than local descent

NTU

Representation in Genetic Algorithms

Binary Encodings

- Chromosomes represented as bit strings: $x \in \{0,1\}^n$
- Historically dominant due to simplicity
- Well suited to feature selection and subset problems
- Bit-flip mutation and single-point crossover are natural operators

Integer and Permutation Encodings

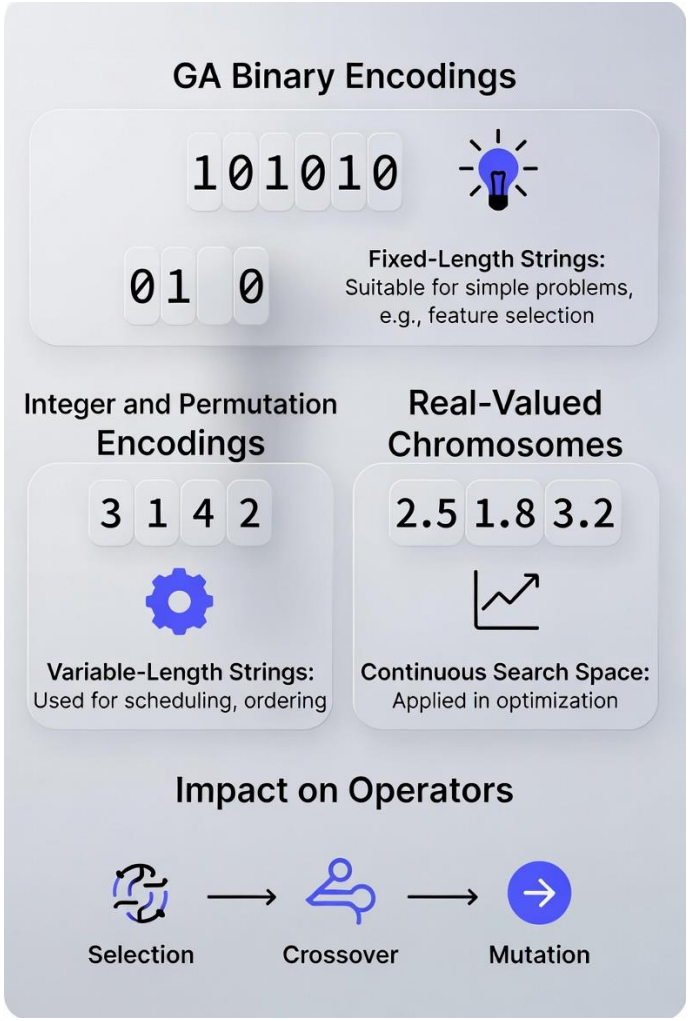
- Chromosomes represent ordered or discrete values
- Permutations preserve element uniqueness and order
- Used in scheduling, routing, and sequencing problems
- Require specialised crossover and mutation (e.g. PMX, swap, inversion)

Real-Valued Chromosomes

- Genes represent continuous parameters: $x \in \mathbb{R}^n$
- Avoid encoding–decoding overhead
- Suitable for numerical optimisation and control problems
- Enable arithmetic crossover and Gaussian mutation

Impact on Operators

- Representation constrains admissible crossover and mutation operators
- Inappropriate operators can violate feasibility or disrupt structure
- Effective GA design aligns encoding, operators, and problem structure



Multi-Objective Fitness and Pareto Dominance

Multi-Objective Optimisation

- Many problems involve conflicting objectives
- Optimise a vector of objectives rather than a single scalar: minimise $F(x) = (f_1(x), f_2(x), \ldots, f_k(x))$
- No single solution optimises all objectives simultaneously

Pareto Dominance

- Solution x_1 dominates x_2 if: x_1 is no worse in all objectives and strictly better in at least one objective
- Non-dominated solutions form the **Pareto front**

Pareto Front

- Represents trade-offs between competing objectives
- Provides a set of optimal compromises rather than one solution
- Decision-making deferred to post-optimisation

Role in Evolutionary Algorithms

- Population-based search naturally approximates Pareto fronts
- Diversity preservation maintains spread across objectives
- Algorithms such as NSGA-II use dominance ranking and crowding

Evolutionary algorithms are particularly well suited to multi-objective optimisation because they return sets of solutions, not a single optimum.

NTU

Pareto Front and Evolutionary Algorithms

Pareto Front

- Set of non-dominated solutions
- Each solution represents a different trade-off
- Improvement in one objective requires degradation in another

Why Evolutionary Algorithms Excel

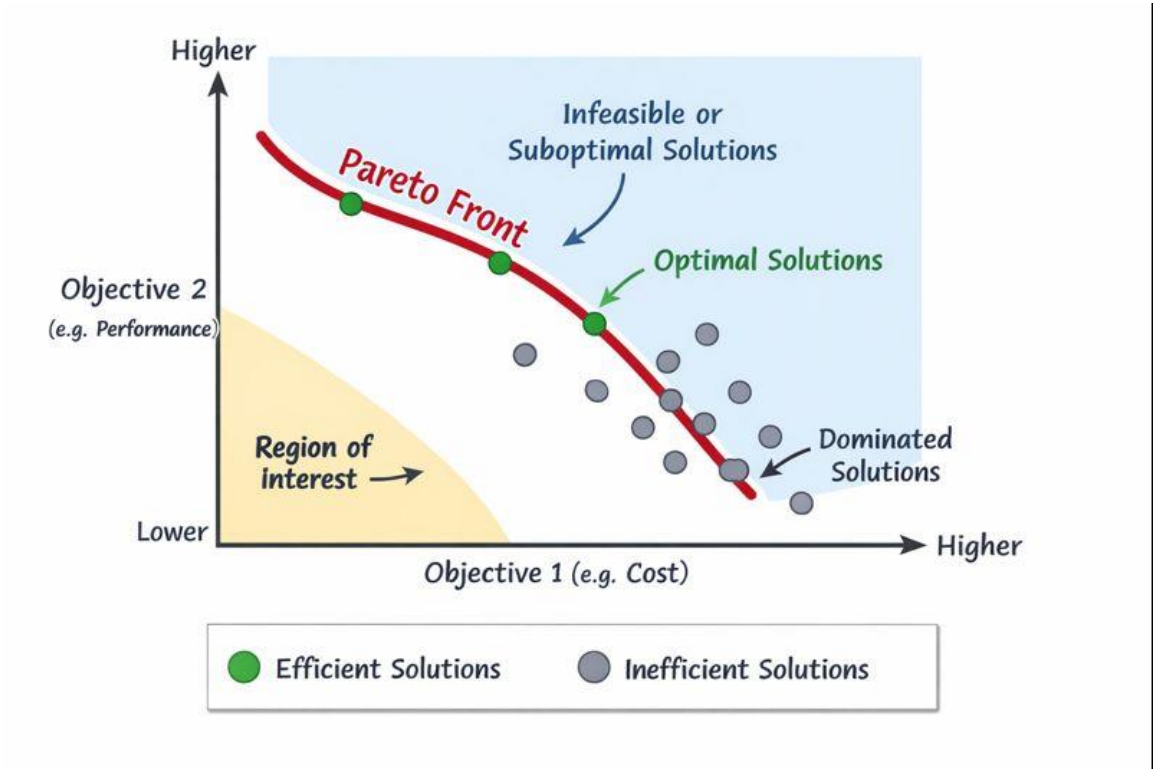
- Population naturally approximates the Pareto front
- Diversity mechanisms preserve spread across objectives
- No need for a priori weighting of objectives

Example Algorithms

- NSGA-II: dominance ranking + crowding distance
- SPEA2: fitness assignment using dominance strength

Key Insight

- Evolutionary multi-objective optimisation supports informed decision-making rather than forcing a single optimum



- two-objective optimisation problem, with Objective 1 on the horizontal axis and Objective 2 on the vertical axis. Each point represents a feasible solution.
- points on the red curve form the Pareto front. Each of these solutions is optimal in the sense that improving one objective would necessarily worsen the other. Grey points are dominated solutions, as better alternatives exist for both objectives.

Selection Mechanisms in Genetic Algorithms

Roulette-Wheel Selection (Fitness-Proportionate)

- Idividuals are selected with probability proportional to their fitness.
- Fitter solutions are more likely, but weaker solutions may still reproduce.
- Simple to implement, but sensitive to fitness scaling and noise.

Tournament Selection

- A random subset of k individuals is sampled; the fittest is selected.
- Selection pressure is explicitly controlled by k.
- Robust, widely used, and independent of fitness scaling.

Rank-Based Selection

- Individuals are ranked by fitness; selection probability depends on rank.
- Prevents domination by extremely fit individuals.
- Provides stable selection pressure, particularly in noisy landscapes.

NTU

Selection Methods: Comparative Overview

Mechanism	Role in GA	Controls Selection Pressure	Strengths	Risks
Roulette	Parent selection	Indirect (fitness scaling)	Simple, probabilistic	Fitness scaling sensitivity
Tournament	Parent selection	Direct (k*)	Robust, easy pressure control	Large k reduces diversity
Rank-based	Parent selection	Stable, moderate	Prevents domination by elites	Slower convergence

**k* is an explicit and intuitive control knob for selection pressure, which is why tournament selection is widely used in practical Genetic Algorithm implementations.

NTU

Selection and the Fitness Landscape

Selection shapes how the population moves across the fitness landscape.

Low selection pressure:

- Population spreads across multiple basins
- Encourages global exploration

Moderate selection pressure:

- Population drifts toward promising basins
- Maintains multiple competing solution

High selection pressure:

- Population collapses into a single basin
- Risk of convergence to local optima

NTU

Elitism: Role, Effects, and GA vs ES Perspective

- **Elitism** preserves the best individual(s) by copying them unchanged into the next generation, guaranteeing that best-so-far fitness never degrades.
- **Effect on convergence -** Elitism accelerates convergence and stabilises progress by preventing the loss of high-quality solutions due to stochastic variation. It is especially useful in noisy or expensive optimisation problems.
- **Risk: loss of diversity -**
  Strong elitism increases effective selection pressure, reduces population diversity, and can cause premature convergence by turning the algorithm into local search.

GA vs ES distinction

- **Genetic Algorithms:** elitism is optional and must be applied conservatively to avoid disrupting recombination and diversity.
- **Evolution Strategies:** elitism is often implicit (μ+λ schemes), where elitism supports controlled, mutation-driven refinement.
- **Design rule:** elitism preserves memory; selection and variation drive search.

NTU

Crossover: Role in Genetic Algorithms

- Recombination of building blocks: Crossover combines parts of two parent solutions, allowing useful partial solutions to be reused rather than rediscovered.

- Schema intuition: Short, well-performing gene patterns are more likely to survive and spread, guiding the search without explicit problem knowledge.

- Search space mixing: Exchanging solution segments creates structured jumps in the search space, enabling efficient global exploration beyond local mutation.

NTU

Crossover Operators in Genetic Algorithms

Single-point crossover:

- A single cut point is selected; genetic material is exchanged after this point.
- Preserves contiguous gene blocks but is sensitive to gene ordering.

Multi-point crossover

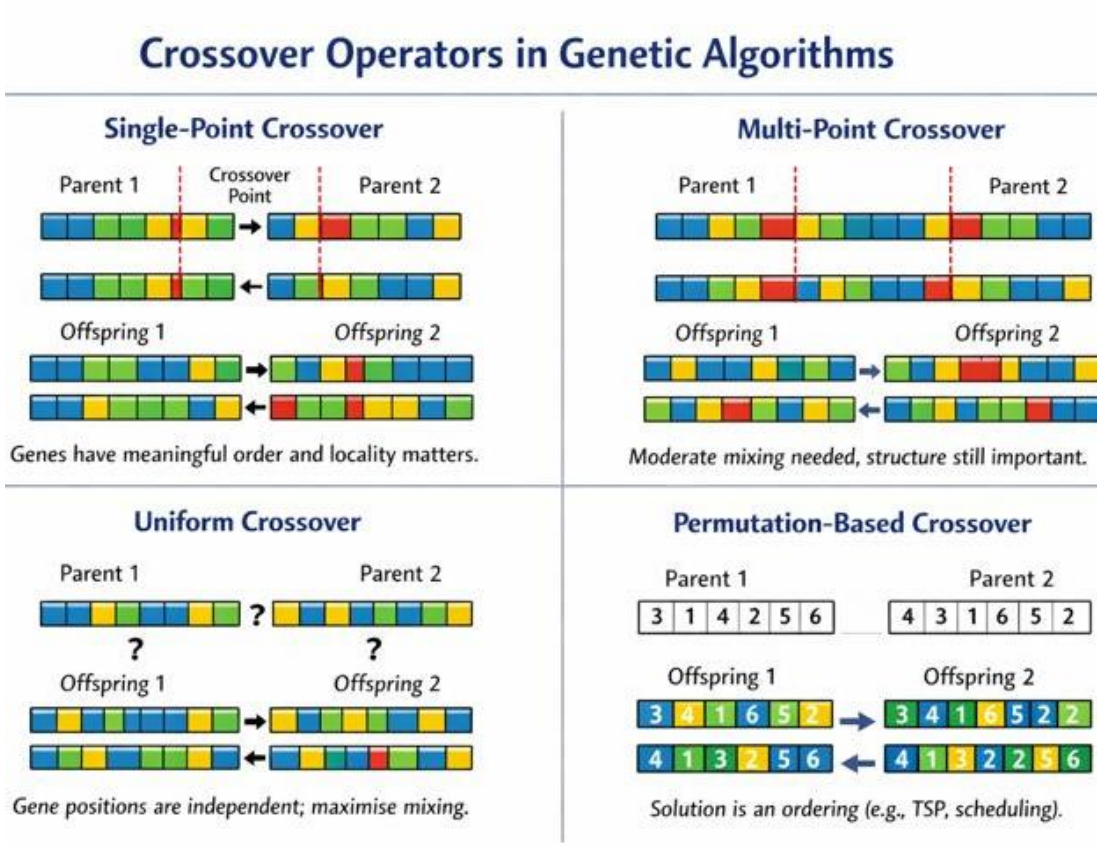
- Multiple cut points are used, alternating segments between parents.
- Increases mixing while retaining some positional structure.

Uniform crossover

- Each gene is independently inherited from either parent with fixed probability.
- Maximises mixing and reduces positional bias, but may disrupt useful structures.

Permutation-based crossover

- Specialised operators (e.g. PMX, OX, CX) preserve feasibility in ordered solutions.
- Essential for problems such as routing, scheduling, and sequencing.



Permutation-Based Crossover Operators

- In permutation problems, each element must appear exactly once. Standard crossover may create invalid solutions with duplicates or missing elements.

PMX – Partially Matched Crossover

- Swaps a segment between parents and repairs remaining positions using a mapping.
- Preserves absolute position information while maintaining feasibility.

OX – Order Crossover

- Copies a segment from one parent and fills remaining positions using the order of the other parent.
- Preserves relative ordering rather than absolute position.

CX – Cycle Crossover

- Identifies cycles between parents and exchanges entire cycles.
- Each gene inherits its position from exactly one parent, with minimal disruption.

Rule of thumb

- Use PMX when positions matter,
- OX when order matters,
- CX when strict positional inheritance is required.

NTU

Mutation in Genetic Algorithms

Mutation introduces random variation into individuals, preventing loss of diversity and enabling exploration of unseen regions of the search space.

Bit-flip mutation

- Individual genes are randomly flipped (e.g. 0 → 1 or 1 → 0).
- Primarily used with binary or discrete encodings to reintroduce lost genetic material.

Swap and inversion mutation

- Swap exchanges the positions of two genes; inversion reverses a contiguous gene segment.
- Preserves feasibility in permutation-based problems such as routing and scheduling.

Mutation rate effects

- Low mutation rates increase the risk of premature convergence.
- High mutation rates destroy useful structures and approximate random search.
- Effective rates balance diversity preservation with exploitation of good solutions.

NTU

Mutation in GA vs ES

Mutation in Genetic Algorithms (GA): is the *secondary operator* used to maintain diversity. It introduces small random changes (e.g. bit-flip, swap, inversion) to prevent premature convergence. Search is driven mainly by **selection and crossover**, with mutation acting as a background exploration mechanism.

Mutation in Evolution Strategies (ES): is the *primary search operator*. New solutions are generated by adding Gaussian noise to real-valued parameters. Mutation strength (step size) is **self-adapted**, allowing the algorithm to adjust its own search scale.

Key contrast

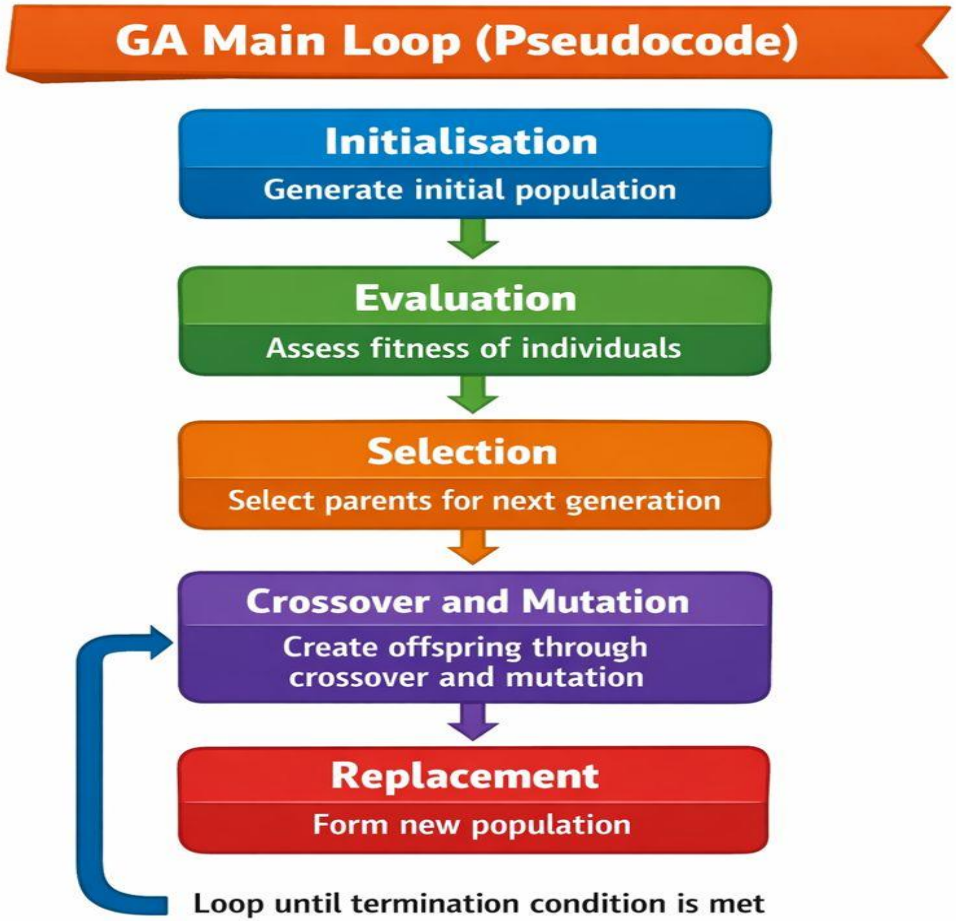
- GA: mutation preserves diversity; crossover drives innovation
- ES: mutation drives innovation; selection controls refinement

Implication

GA excels in structured and combinatorial spaces,
ES excels in continuous, noisy, and ill-conditioned optimisation.

NTU

GA Main Loop


GA Main Loop (Pseudocode)
Initialisation
Generate initial population
Evaluation
Assess fitness of individuals
Selection
Select parents for next generation
Crossover and Mutation
Create offspring through crossover and mutation
Replacement
Form new population
Loop until termination condition is met

Termination Criteria in Genetic Algorithms

Maximum number of generations

- The algorithm stops after a predefined number of generations.
- Provides a simple and predictable upper bound on runtime, independent of solution quality.

Fitness convergence

- Termination occurs when improvement in fitness falls below a threshold over several generations.
- Indicates that the population has stabilised and further search is unlikely to yield significant gains.

Computational budget

- The algorithm stops when a time limit or maximum number of fitness evaluations is reached.
- Essential for expensive or real-time optimisation problems where evaluation cost dominates runtime.

NTU

Steady-State vs Generational Genetic Algorithms

Population update strategies

- In a generational GA, the entire population is replaced at each generation by newly created offspring.
- In a steady-state GA, only a small number of individuals are replaced at a time, with offspring immediately inserted into the population.

Convergence speed

- Steady-state GAs often converge faster in wall-clock time because improvements are incorporated immediately.
- Generational GAs progress in discrete steps and may converge more slowly but offer clearer generational dynamics.

Diversity implications

- Generational replacement maintains diversity more naturally, as many individuals are replaced simultaneously.
- Steady-state replacement increases selection pressure and can reduce diversity if replacement is too aggressive.

Design trade-off

- Generational GAs favour exploration and population-level diversity,
- Steady-state GAs favour exploitation and rapid refinement of solutions.

NTU

Diversity Preservation in Genetic Algorithms

Genetic Algorithms operate on populations, and their effectiveness depends on maintaining a diverse set of candidate solutions. Loss of diversity leads to premature convergence and poor performance on complex or multimodal problems.

Niching and fitness sharing

- Niching methods encourage the formation of subpopulations around different regions of the search space.
- Fitness sharing reduces an individual's effective fitness when many similar individuals exist nearby, discouraging overcrowding of a single solution basin and promoting parallel exploration of multiple optima.

Crowding methods

- Crowding replaces individuals with similar offspring rather than the worst individuals globally.
- By enforcing local competition, crowding preserves multiple distinct solutions and prevents dominant individuals from overtaking the entire population.

Multimodal optimisation

- In landscapes with multiple local and global optima, diversity preservation allows the algorithm to locate and maintain several high-quality solutions simultaneously.
- This is essential when the global optimum is unknown, deceptive, or when multiple acceptable solutions exist.

Diversity mechanisms transform a GA from a single-solution optimiser into a population-based search capable of exploring and maintaining multiple optima in parallel.

NTU

Constraint Handling in Genetic Algorithms

Many optimisation problems impose constraints that restrict the feasible solution space. Genetic Algorithms, by default, generate unconstrained solutions and therefore require explicit mechanisms to guide the search toward feasibility.

Penalty methods

- Constraints are incorporated into the fitness function by penalising infeasible solutions.
- The fitness of a solution is reduced in proportion to the degree of constraint violation, discouraging infeasible regions while still allowing exploration near constraint boundaries.
- Penalty strength must be carefully tuned; overly strong penalties prevent exploration, while weak penalties allow infeasible dominance.

Repair operators

- Infeasible solutions are transformed into feasible ones using problem-specific rules.
- Repair operators preserve genetic material while enforcing constraints, improving search efficiency but requiring domain knowledge and additional implementation effort.

Feasibility rules

- Selection is biased toward feasible solutions, for example by always preferring feasible over infeasible individuals, or by ranking feasible solutions first.
- Such rules simplify constraint handling and reduce parameter tuning, but may restrict exploration in early generations.

Design trade-off

- Penalty methods favour generality, repair operators favour efficiency, and feasibility rules favour simplicity.
- Effective constraint handling balances feasibility enforcement with continued exploration.

NTU

Memetic Algorithms

GA and local search

- Memetic Algorithms combine a Genetic Algorithm with a local search method applied to individuals, typically after crossover and mutation.
- The GA performs global exploration, while local search refines solutions by exploiting nearby regions of the search space.

Hybrid optimisation

- This hybrid approach leverages the complementary strengths of both methods:
- the GA maintains population diversity and explores multiple regions in parallel,
- while local search accelerates convergence by efficiently improving candidate solutions.
- Local improvement can be applied to all individuals or selectively to elite or newly generated offspring.

When hybridisation helps

- Memetic Algorithms are particularly effective when the fitness landscape is complex but locally smooth, allowing local search to make rapid improvements.
- They are well suited to combinatorial optimisation, constrained problems, and scenarios where high-quality solutions are required within limited computational budgets.
- Hybridisation is less effective when local search is expensive or when the landscape lacks exploitable local structure.

Memetic Algorithms balance global exploration with intensive local exploitation, often achieving higher solution quality than pure evolutionary or local search methods alone.

NTU

Multi-Objective Genetic Algorithms

Many real-world problems involve conflicting objectives that cannot be reduced to a single scalar fitness function. Multi-objective Genetic Algorithms aim to optimise several objectives simultaneously without imposing arbitrary weightings.

- **Pareto dominance: A solution is said to** *Pareto-dominate* another if it is no worse in all objectives and strictly better in at least one.
  The set of non-dominated solutions forms the **Pareto front**, representing the best achievable trade-offs between objectives.
- **Trade-offs instead of a single optimum:** MOGAs do not return one “best” solution. Instead, they maintain a diverse population of solutions along the Pareto front, allowing decision-makers to choose solutions that best match external preferences or constraints.
- **Applications:** Multi-objective GAs are widely used in engineering design, feature selection, scheduling, energy optimisation, and control systems, where objectives such as performance, cost, robustness, and efficiency must be balanced simultaneously.
- *MOGAs transform optimisation from finding a single answer into exploring a structured set of optimal compromises.*

NTU

Non-dominated Sorting Genetic Algorithm II (NSGA-II)

Non-dominated sorting

- The population is partitioned into successive Pareto fronts based on dominance.
- Solutions in the first front are non-dominated; subsequent fronts contain increasingly dominated solutions.
- Selection prioritises lower-rank fronts, ensuring progress toward the Pareto-optimal set.

Crowding distance

- Within each Pareto front, solutions are ranked by crowding distance, a measure of local solution density in objective space.
- Solutions in sparsely populated regions are preferred, promoting diversity and uniform coverage of the Pareto front without additional parameters.

Why NSGA-II works

- NSGA-II combines elitism, dominance-based selection, and diversity preservation in a computationally efficient framework.
- It avoids arbitrary objective weighting, scales well with population size, and reliably approximates well-distributed Pareto fronts in a single run.

NSGA-II balances convergence toward optimal trade-offs with diversity across objectives, making it one of the most widely used multi-objective evolutionary algorithms.

NTU

Evolution Strategies: Overview

Continuous optimisation focus

- Evolution Strategies are designed primarily for optimisation in continuous, real-valued parameter spaces.
- They are particularly effective for non-linear, noisy, ill-conditioned, or non-differentiable problems where gradient-based methods are unreliable or unavailable.

Mutation-driven search

- Unlike Genetic Algorithms, Evolution Strategies rely almost entirely on mutation as the search operator.
- New candidate solutions are generated by adding stochastic perturbations to existing solutions, with the magnitude and direction of mutation often adapted automatically during the search.
- Selection acts mainly to retain improved solutions rather than to recombine genetic material.

European research lineage

- Evolution Strategies originated in Europe in the 1960s and 1970s through the work of researchers such as Rechenberg and Schwefel, with an engineering-oriented focus.
- Their development was motivated by real-world optimisation problems in aerodynamics and control, emphasising empirical performance, robustness, and minimal assumptions about the objective function.

- Evolution Strategies treat optimisation as an adaptive stochastic process over continuous spaces, prioritising robustness and self-adaptation over structural recombination.

NTU

Representation in Evolution Strategies

Real-valued vectors

- In Evolution Strategies, each individual is represented directly as a vector of real-valued parameters.
- Each component corresponds to a decision variable of the optimisation problem, avoiding artificial gene encodings.

No encoding or decoding

- Unlike Genetic Algorithms, ES do not require binary or symbolic representations.
- This eliminates encoding–decoding overhead and prevents representational bias, allowing the search to operate directly in the problem's native space.

Direct parameter optimisation

- Mutation and adaptation act directly on the parameters being optimised, enabling smooth, continuous exploration of the search space.
- This makes ES particularly suitable for tuning control parameters, model weights, and hyperparameters in continuous domains.

By aligning the representation with the problem space, Evolution Strategies achieve efficient, robust optimisation without relying on problem-specific encodings.

NTU

Elitism in Evolution Strategies: (μ,λ) vs (μ+λ)

Aspect	(μ,λ) Evolution Strategy	(μ+λ) Evolution Strategy
Parent population	μ	μ
Offspring generated	λ (λ > μ)	λ (λ > μ)
Selection pool	Offspring only	Parents ∪ offspring
Elitism	No	Yes (implicit)
Best-so-far survival	Not guaranteed	Guaranteed
Selection pressure	Lower	Higher
Exploration	Strong	Moderate
Exploitation	Limited	Strong
Diversity	Better preserved	Reduced over time
Convergence speed	Slower, less stable	Faster, more stable
Risk of premature convergence	Low	Higher
Typical use	Multimodal or deceptive landscapes	Refinement and local optimisation

NTU

λ is the number of new candidate solutions are created in each generation before choosing which ones survive.

Mutation in Evolution Strategies

Gaussian perturbations

- New candidate solutions are generated by adding random noise drawn from a Gaussian distribution to the current solution vector.
- This produces smooth, continuous variations and allows controlled exploration of the search space.

Step-size control

- The standard deviation of the Gaussian mutation determines the step size.
- Large step sizes promote global exploration, while small step sizes enable fine-grained local refinement.
- In modern ES, step sizes are often self-adapted, allowing the algorithm to adjust its search behaviour automatically.

Exploration radius

- The step size effectively defines the radius of exploration around each solution.
- As the search progresses, this radius typically shrinks, focusing the search on promising regions while maintaining robustness to noise and local irregularities.

In ES, mutation is not a background operator but the primary mechanism that controls both exploration and exploitation through adaptive step sizes.

NTU

Self-Adaptation in Evolution Strategies

Strategy parameters in the genome

- In Evolution Strategies, individuals encode not only solution variables but also strategy parameters, such as mutation step sizes.
- These parameters evolve alongside the solution itself, allowing the algorithm to modify its own search behaviour.

Evolution of mutation strength

- Mutation step sizes are subject to variation and selection in the same way as solution parameters.
- Individuals with well-tuned mutation strengths are more likely to produce successful offspring and therefore propagate their strategy parameters.
- As a result, the algorithm automatically adjusts how aggressively it explores or exploits the search space.

Why self-adaptation matters

- Self-adaptation removes the need for manual parameter tuning and allows the algorithm to respond dynamically to different regions of the fitness landscape.
- It enables large exploratory steps early in the search and smaller, precise steps near optima, improving robustness and efficiency across diverse problem settings.

Self-adaptation allows Evolution Strategies to learn how to search while solving the optimisation problem, rather than relying on fixed, externally chosen parameters.

NTU

CMA-ES: Motivation

Ill-conditioned problems

- In many optimisation problems, progress is much easier in some directions than others.
- Simple Evolution Strategies with fixed or diagonal step sizes struggle when the fitness landscape contains narrow valleys or steep ridges, leading to slow convergence.

Correlated variables

- Real-world problems often involve strong dependencies between variables.
- When parameters are correlated, independent mutations in each dimension are inefficient because useful search directions do not align with coordinate axes.

Limitations of simple ES

- Isotropic or diagonal mutation assumes either uniform sensitivity or independent variables.
- Such assumptions cause inefficient zig-zagging behaviour and poor scaling when the problem is rotated or poorly scaled.

CMA-ES overcomes these limitations by learning a full covariance matrix that captures variable correlations and landscape geometry, enabling efficient, adaptive search in complex continuous spaces.

CMA-ES: Core Idea

Learning the covariance matrix

- CMA-ES maintains and updates a covariance matrix that describes the shape and orientation of the search distribution.
- Rather than mutating parameters independently, the algorithm learns how variables interact by observing which mutations lead to successful improvements.

Principal search directions

- The covariance matrix encodes dominant directions in which high-quality solutions are found.
- By stretching and rotating the search distribution, CMA-ES aligns mutations with valleys, ridges, and curved structures in the fitness landscape, enabling efficient movement toward optima.

Second-order information without gradients

- CMA-ES implicitly approximates curvature information similar to a second-order optimiser, but without requiring gradients or Hessians.
- This makes it robust to noise, discontinuities, and black-box objectives where analytical information is unavailable.

CMA-ES adapts where and how it searches by learning the geometry of the problem directly from sampled solutions, achieving near–second-order performance using only function evaluations.

NTU

CMA-ES: Update Intuition

Rank-based updates

- CMA-ES updates its internal parameters using only the relative ranking of solutions, not their absolute fitness values.
- This makes the algorithm invariant to monotonic transformations of the objective function and robust to noise and scaling effects.

Evolution paths

- Evolution paths accumulate information about successive successful search steps over multiple generations.
- They act as a form of momentum, capturing persistent directions of improvement and enabling the algorithm to distinguish meaningful trends from random fluctuations.

Step-size adaptation

- CMA-ES continuously adapts its global step size based on the length and consistency of evolution paths.
- Consistent progress leads to larger steps to speed up exploration, while oscillatory or stagnant behaviour reduces step size to enable fine-grained refinement near optima.

CMA-ES updates combine short-term ranking information with long-term directional memory, allowing the algorithm to adapt both the shape and scale of its search automatically.

NTU

Differential Evolution (DE)

Vector difference mutation

- Differential Evolution generates new candidate solutions by adding the scaled difference between two population vectors to a third vector.
- This mechanism automatically adapts step sizes to population spread, enabling self-scaled exploration without explicit mutation parameters.

Relation to Evolution Strategies

- DE operates in continuous spaces like ES and is mutation-driven rather than recombination-driven like GA.
- However, instead of Gaussian perturbations, DE uses population differences to guide search directions, implicitly capturing problem scale and orientation.
- In this sense, DE can be viewed as a population-based, self-scaling alternative to classical ES.

When DE is preferable

- DE is particularly effective for low- to medium-dimensional continuous optimisation problems with smooth but multimodal landscapes.
- It performs well when gradients are unavailable and when minimal parameter tuning is desired.
- DE is often preferred when CMA-ES is computationally too expensive or when a simpler, robust optimiser is sufficient.

Differential Evolution exploits population geometry directly, using relative differences between solutions to drive efficient, adaptive search.

NTU

Genetic Programming (GP)

Tree-based representations

- Genetic Programming represents solutions as tree structures rather than fixed-length vectors or strings.
- Internal nodes correspond to functions or operators, while leaf nodes represent terminals such as variables or constants.
- This representation allows GP to naturally express hierarchical and variable-length solutions.

Program evolution

- GP evolves entire programs or symbolic expressions by applying evolutionary operators to trees.
- Crossover exchanges subtrees between parent programs, while mutation modifies nodes or subtrees.
- Fitness is evaluated based on program behaviour, such as prediction accuracy, control performance, or symbolic correctness.

Brief contrast with GA and ES

- Genetic Algorithms evolve fixed-length encodings of candidate solutions, focusing on parameter or component optimisation.
- Evolution Strategies optimise real-valued parameter vectors using mutation-driven search.
- Genetic Programming differs fundamentally by evolving structure and logic, making it suitable for discovering models, rules, or algorithms rather than tuning parameters.

Genetic Programming treats optimisation as the evolution of executable structures, enabling automatic discovery of symbolic models and programs beyond fixed representations.

NTU

Co-evolutionary Algorithms

Competitive and cooperative evolution

- Co-evolutionary algorithms involve multiple populations that evolve simultaneously and interact with one another.
- In competitive co-evolution, populations represent opposing entities whose fitness depends on outperforming others (e.g. predator–prey, attacker–defender).
- In cooperative co-evolution, populations represent complementary components that must work together to form a complete solution.

Multi-population dynamics

- Each population evolves under its own selection and variation processes, but fitness evaluation depends on interactions with individuals from other populations.
- This creates dynamic fitness landscapes, where improvement in one population changes the optimisation pressure on others, often leading to continual adaptation rather than static convergence.

Example scenarios

- Competitive co-evolution is used in games, adversarial learning, security, and strategy optimisation.
- Cooperative co-evolution is applied to large-scale optimisation, neural network decomposition, multi-agent systems, and modular problem solving, where different populations evolve separate subcomponents.

Co-evolution shifts optimisation from improving isolated solutions to improving interacting systems, enabling scalable, modular, and adversarial learning that single-population algorithms cannot capture.

NTU

Evolutionary Feature Selection

Combinatorial optimisation

- Feature selection is inherently a combinatorial problem, as the number of possible feature subsets grows exponentially with the number of features.
- Evolutionary algorithms efficiently explore this large, discrete search space without requiring exhaustive enumeration or gradient information.

Wrapper methods

- In wrapper-based feature selection, each candidate solution represents a subset of features that is evaluated using a learning algorithm.
- The evolutionary algorithm searches for subsets that maximise predictive performance, capturing feature interactions that filter-based methods often miss.
- Although computationally expensive, wrapper methods typically achieve higher accuracy.

Multi-objective trade-offs

- Evolutionary feature selection naturally supports multi-objective optimisation, balancing competing goals such as classification accuracy, model complexity, and computational cost.
- Rather than producing a single solution, the algorithm can return a Pareto front of feature subsets, allowing informed trade-offs between performance and simplicity.

Evolutionary feature selection treats model design as an optimisation problem over feature subsets, enabling flexible, high-quality solutions in high-dimensional settings.

NTU

Hyperparameter Optimisation

Black-box tuning

- Hyperparameter optimisation is a black-box problem where gradients are unavailable and evaluations are expensive.
- Evolutionary methods operate directly on performance feedback, making no assumptions about smoothness, convexity, or differentiability.

GA and ES in AutoML

- Genetic Algorithms are well suited to discrete and mixed hyperparameters, such as architecture choices, feature subsets, and categorical settings.
- Evolution Strategies excel at continuous hyperparameters, such as learning rates, regularisation strengths, and optimiser parameters.
- Both are widely used in AutoML pipelines for joint optimisation of model structure and training parameters.

Comparison with Bayesian optimisation

- Bayesian optimisation is sample-efficient and effective in low-dimensional spaces but struggles with high-dimensional, mixed, or conditional search spaces.
- GA and ES scale better to complex configurations, support parallel evaluation naturally, and are more robust to noisy or unstable training processes.

Bayesian optimisation is efficient for small, smooth problems, while evolutionary methods provide flexibility and robustness for large-scale, heterogeneous AutoML search spaces.

NTU

GA vs ES vs PSO vs DE vs Bayesian Optimisation (BO)

Aspect	Genetic Algorithms (GA)	Evolution Strategies (ES)	Particle Swarm Optimisation (PSO)	Differential Evolution (DE)	Bayesian Optimisation (BO)
Search paradigm	Evolutionary recombination	Mutation-driven stochastic search	Social learning and velocity updates	Population-difference mutation	Surrogate-based probabilistic search
Parameter types	Discrete, categorical, mixed	Continuous, real-valued	Continuous (adaptable to discrete)	Continuous, real-valued	Mostly continuous
Handling mixed spaces	Strong	Moderate	Weak–moderate	Weak	Weak
Primary variation mechanism	Crossover	Gaussian mutation	Velocity update toward best solutions	Vector difference mutation	Acquisition-driven sampling
Adaptivity	Manual operator tuning	Self-adaptive step sizes / covariance	Adaptive social coefficients	Self-scaled via population spread	Model-driven
Sample efficiency	Moderate	Moderate	Moderate	Moderate–good	High (low dimensions)
Scalability (dimensions)	Good	Moderate	Good	Moderate	Poor–moderate
Noise robustness	High	High	Moderate	High	Moderate
Parallel evaluation	Natural	Natural	Natural	Natural	Limited
Hyperparameter interactions	Captured implicitly	Captured via mutation geometry	Weakly captured	Implicitly captured	Explicitly modelled (costly)
Typical AutoML role	Architecture / structure search	Continuous tuning and refinement	Fast coarse tuning	Robust continuous tuning	Fine-tuning few parameters
Key strength	Flexibility and diversity	Robust continuous optimisation	Speed and simplicity	Self-scaling mutations	Sample efficiency
Key limitation	Slower convergence	Computational cost	Premature convergence	Parameter sensitivity	Poor scalability

Neuroevolution

Evolving network weights

- Neuroevolution can optimise neural network weights using evolutionary algorithms instead of gradient-based learning.
- This is particularly useful when gradients are unavailable, noisy, or when the objective is non-differentiable, such as in reinforcement learning or simulation-based environments.

Evolving network architectures

- Beyond weights, neuroevolution can evolve network topologies, including the number of layers, connections, and activation functions.
- This allows the automatic discovery of network structures rather than relying on manually designed architectures.

Relation to Neural Architecture Search (NAS)

- Neuroevolution is a population-based form of NAS, where architectures are explored through mutation and recombination rather than surrogate models or gradient-based relaxations.
- Evolutionary NAS methods naturally handle discrete design choices, conditional structures, and multi-objective trade-offs such as accuracy versus model size or energy consumption.

Neuroevolution treats neural network design as an evolutionary optimisation problem, enabling flexible, gradient-free learning of both structure and parameters.

NTU

Evolutionary Neural Architecture Search (NAS)

Search space challenges

- Neural architecture spaces are extremely large, discrete, hierarchical, and often conditional.
- Design choices such as layer types, connectivity patterns, and hyperparameters interact non-linearly, making exhaustive or gradient-based search impractical.

Population-based NAS

- Evolutionary NAS maintains a population of candidate architectures that are iteratively mutated, recombined, and selected based on performance.
- Population diversity enables simultaneous exploration of multiple architectural patterns, while selection drives convergence toward high-performing designs.
- Multi-objective optimisation is naturally supported, allowing trade-offs between accuracy, latency, model size, and energy consumption.

Why evolutionary computation is competitive

- Evolutionary methods require no differentiability, surrogate accuracy, or search space relaxation.
- They scale well to mixed and discrete design variables, support parallel evaluation, and are robust to noisy or unstable training processes.
- As a result, evolutionary NAS remains competitive for large, constrained, or hardware-aware architecture search problems.

Evolutionary NAS frames architecture design as a robust, population-based optimisation problem, well suited to complex, real-world constraints beyond pure accuracy.

NTU

Evolutionary Computation for General-Purpose AI

Adaptation over fixed optimisation

- Evolutionary Computation emphasises continual adaptation rather than optimisation toward a single static objective.
- This makes EC well suited to open, non-stationary environments where tasks, constraints, or goals change over time.

Multi-task and continual learning

- Populations can encode multiple behaviours, skills, or solutions simultaneously, enabling parallel learning across tasks.
- Knowledge can be reused, recombined, or repurposed through evolutionary processes, supporting transfer and lifelong learning without catastrophic forgetting.

Population diversity as robustness

- Maintaining a diverse population provides inherent robustness to uncertainty, noise, and unforeseen scenarios.
- Rather than relying on a single optimal model, EC maintains a repertoire of viable solutions, increasing resilience to failures and distribution shifts.

Evolutionary Computation supports general-purpose intelligence by prioritising adaptability, diversity, and resilience, offering a complementary paradigm to single-model, task-specific learning systems.

NTU

When to Use Genetic Algorithms

Discrete search spaces

- Genetic Algorithms are well suited to problems where decision variables are discrete, categorical, or symbolic.
- They avoid the need for gradient information and can naturally explore large, irregular search spaces.

Combinatorial structure

- GA perform particularly well when solutions are composed of interacting components that can be recombined.
- Crossover exploits problem structure by mixing partial solutions, making GA effective for routing, scheduling, feature selection, and configuration problems.

Multiple conflicting objectives

- GA naturally extend to multi-objective optimisation, maintaining populations of solutions that represent different trade-offs.
- Methods such as NSGA-II enable simultaneous optimisation without reducing objectives to a single scalar fitness.

NTU

When to Use Evolution Strategies

Continuous parameters

- Evolution Strategies are specifically designed for optimisation in continuous, real-valued parameter spaces.
- They operate directly on parameter vectors without encoding, making them efficient and natural for tuning real-valued models.

Noisy or ill-conditioned landscapes

- ES are robust to noise, discontinuities, and poorly scaled objective functions.
- Adaptive mutation and covariance learning allow ES to navigate narrow valleys, ridges, and correlated dimensions where gradient-based methods struggle.

Expensive or unavailable gradients

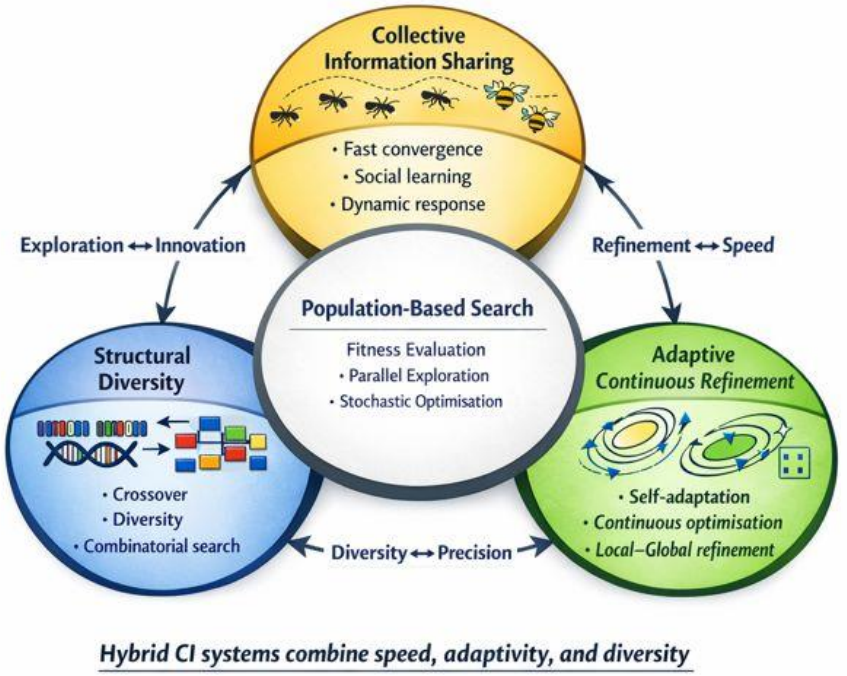
- ES do not rely on gradient information and therefore remain effective when gradients are costly to compute, unreliable, or entirely unavailable.
- This makes ES suitable for simulation-based optimisation, physical systems, and black-box objectives.

Evolution Strategies excel when reliable gradients cannot be assumed and robustness is more important than strict computational efficiency.

NTU

GA, ES and Swarm Algorithms: Complementarity

- Swarm: fast collective exploration and exploitation
- ES: adaptive refinement in continuous spaces
- GA: diversity preservation and structural innovation
- Hybridisation balances speed, precision, and robustness



Hybrid CI systems combine speed, adaptivity, and diversity

Conclusion

GA and ES are complementary, not competing

- Genetic Algorithms and Evolution Strategies address different aspects of optimisation.
- GA excel at exploiting structure, discreteness, and recombination of solution components, while ES specialise in adaptive, continuous refinement through mutation-driven search.
- Effective Computational Intelligence systems increasingly combine both.

Design choices dominate performance

- Algorithm performance depends more on representation, operators, selection pressure, and parameter adaptation than on the choice of algorithm family alone.
- Poorly designed GA or ES implementations will underperform simpler methods, while well-designed variants can outperform state-of-the-art alternatives.

No universal optimiser

- No single optimisation algorithm performs best across all problems.
- Performance is inherently problem-dependent, shaped by landscape structure, noise, dimensionality, and constraints.
- Successful optimisation requires matching algorithm design to problem characteristics rather than defaulting to a single method.

Evolutionary Computation is a flexible toolkit, not a one-size-fits-all solution.

NTU

Quiz (1 / 3)

1. Which component primarily drives search in Evolution Strategies?

- A. Crossover
- B. Selection
- C. Mutation
- D. Fitness scaling

2. Why is crossover central to Genetic Algorithms but not to Evolution Strategies?

- A. ES cannot represent discrete solutions
- B. GA exploit recombination of building blocks
- C. ES use larger populations
- D. GA do not use mutation

NTU

Quiz (2 / 3)

- 3. What problem does elitism primarily address?
- A. Slow convergence
- B. Loss of best-so-far solutions
- C. High computational cost
- D. Noisy fitness functions

- 4. In multi-objective optimisation, what does the Pareto front represent?
- A. The single best solution
- B. The average population fitness
- C. The set of non-dominated trade-off solutions
- D. The solution with minimum cost

NTU

Quiz (3 / 3)

5. Which method is most likely to stagnate quickly without diversity control?

- A. Genetic Algorithms
- B. Evolution Strategies
- C. Particle Swarm Optimisation
- D. Bayesian Optimisation

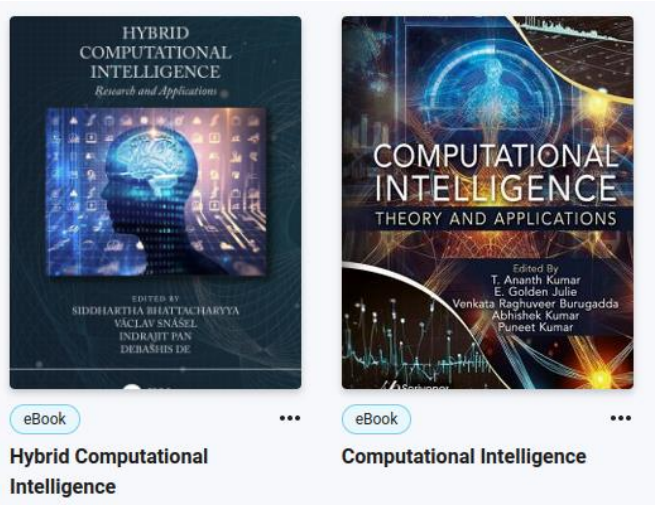
6. When would Bayesian Optimisation be preferred over GA or ES?

- A. High-dimensional mixed search spaces
- B. Discrete combinatorial problems
- C. Expensive evaluations with few continuous parameters
- D. Noisy, non-stationary environments

NTU

References

[1] D. Molina *et al.*, "Evolutionary Computation for the Design and Enrichment of General-Purpose Artificial Intelligence Systems: Survey and Prospects," in *IEEE Transactions on Evolutionary Computation*, doi: 10.1109/TEVC.2025.3530096.
[2] Taha, Z.Y., Abdullah, A.A. & Rashid, T.A. Optimizing feature selection with genetic algorithms: a review of methods and applications. Knowl Inf Syst 67, 9739–9778 (2025). https://doi.org/10.1007/s10115-025-02515-1



Nottingham Trent University

Department of Computer Science

Summary

In this lecture you learnt:

- Computational Intelligence and Evolutionary Computation
- Motivation: Black-Box, Non-Convex, and Noisy Optimisation
- Evolutionary Computation: Core Principles and Taxonomy
- Genetic Algorithms
  - Representation, Selection, Crossover, Mutation
  - Advanced and Multi-Objective Variants
- Evolution Strategies
  - Self-Adaptation and Continuous Optimisation
  - CMA-ES
- Related Population-Based Methods
  - Differential Evolution
  - Genetic Programming
- Comparative Analysis and Complementarity
  - GA vs ES vs Swarm Methods
  - Hybrid Approaches
- Modern Applications and Research Directions

This lecture contributes to the following learning outcomes:

MLO1 - Demonstrate a comprehensive understanding of the core principles, algorithms, and applications of computational intelligence.

MLO3 - Apply computational intelligence methods to real-world problems and interpret the results effectively

MLO7 - Demonstrate critical thinking and creativity in designing and implementing innovative computational intelligence solutions.

Nottingham Trent University
Department of Computer Science

COMP40771 – Computational Intelligence

Lecture 3: Evolutionary Computation and Genetic Algorithms

Dr Pedro Machado pedro.machado@ntu.ac.uk